# COMPSCI 701:
# Special Topic: Creating Maintainable Software

## Lecture 1.2: Object-Oriented Design and Maintainability

Yu-Cheng Tu, Ewan Tempero
School of Computer Science

# Agenda

- Admin
  - Schedule Office hours
  - Class Rep reminder
  - Assignment 1?
- Finish lecture 1.1
- Object-oriented design — finding maintainable classes

# Finish Lecture 1.1

## Class (OOP)

- A concept associated with the “object-oriented paradigm”
- The basis for “object-oriented design”
- Most languages that claim to be “object-oriented” have some form of “class” construct
- One kind of decision that needs to be made is what classes to have in a design
- Does the choice of class affect the maintainability of the design?
- Yes! (but why?)

## OO Language Features?

- Many discussions about “OO” are mostly about the characteristics of the language
- Can we talk about object-oriented design independent of any programming language?
- What features that people talk about for OO languages must we have?

# Language features

To be considered “OOP”, which of the following must a language have?

- objects (naturally!)
- encapsulation (what is encapsulated? $\Rightarrow$ objects)
- classes (what about Self?, Javascript? — prototype-based inheritance)
- inheritance (Emerald? *depends what you mean by inheritance*) not a requirement
- types (Smalltalk?)
- generic types (Smalltalk?, Java before generics?)
- polymorphism, (a.k.a virtual function call, dynamic dispatch, message send)

> “Working in an object-oriented language [...] is neither a necessary or sufficient condition for doing object-oriented programming. . . ” Timothy Budd 2001 “An Introduction to Object-Oriented Programming (3rd Ed)” Pearson

## Language features

To be considered "OOP", which of the following must a language have?

- objects (naturally!)
- encapsulation (what is encapsulated? $\Rightarrow$ objects)
- classes (what about Self?, Javascript? — prototype-based inheritance)
- inheritance (Emerald? *depends what you mean by inheritance*)
- types (Smalltalk?)
- generic types (Smalltalk?, Java before generics?)
- polymorphism, (a.k.a virtual function call, dynamic dispatch, message send)
- Objects and polymorphism are **run-time** concepts

> "Working in an object-oriented language [...] is neither a necessary or sufficient condition for doing object-oriented programming. . . " Timothy Budd 2001 "An Introduction to Object-Oriented Programming (3rd Ed)" Pearson

# Definition: Object-oriented design

- An *object-oriented program* is one that **when it executes** creates objects that send messages to each other.

  > "... the most important aspect of OOP is the creation of a universe of largely autonomous interacting agents." Timothy Budd 2001 "An Introduction to Object-Oriented Programming (3rd Ed)" Pearson

- An object-oriented design is one that describes an object-oriented program.

## Consequences

- encapsulation is a consequence of interaction between objects only being by message sent
  - . . . and so direct access to fields is “not object-oriented”
- inheritance (in any form) is not relevant to what an object-oriented program is — it *is* relevant to design quality however (avoidance of duplicate code)
- types are about preventing certain kinds of faults
- Static members are not part of an object-oriented design
- classes are for describing and creating objects
- $\Rightarrow$ deciding what objects there should be is fundamental to creating a maintainable object-oriented design

# Object-oriented design maintainability

- Are the objects “good”?
- Are the messages “good”?

$\Rightarrow$ What is an “object” and what makes it good?

$\Rightarrow$ What is a “message” and what makes it good?

# Objects

- Traditionally, an “object” is described as something that has identity, state, and behaviour

  (e.g. Grady Booch. 2004. Object-Oriented Analysis and Design with Applications (3rd Ed). Addison Wesley)
- If an object-oriented program creates objects that send messages to each other then:
  1. for one object to send a message to another, it must have some way of identifying that other object
  2. in order for sending a message to be useful, the consequences of the message must provide value to the sender. Consequences depend on:
     - how the receiving object responds or behaves upon receiving the message
     - the history or state of the receiving object

# Finding Good Objects

- Is it sufficient to just use any old objects, or are some objects better than others?
- Heuristic 3.6. "Model the real world whenever possible." Arthur J. Riel (1996) "Object-Oriented Design Heuristics" Addison-Wesley
- "Identify responsibilities [behaviour], group them into roles, define collaboration patterns between roles, and then identify the objects that can play these roles"

  responsibility = an obligation to perform a task or know information

  Christensen (2005) "Implications of perspective in teaching objects first and object design." ITiCSE'05, `https://doi.org/10.1145/1151954.1067474`

  Summarising Budd's presentation of Responsibility-Driven Design (Wirfs-Brock, Wilkerson, Wiener (1990) "Designing Object-Oriented Software" Prentice-Hall)
- "This is especially true if we take an object-oriented view of the world, because objects, as abstractions of entities in the real world, represent a particularly dense and cohesive clustering of information." Booch
- Look at a description of what needs to be built and identify nouns and verbs, because "... the nouns will often correspond to classes and objects, whereas verbs will correspond to the things those object do..." Barnes, Kölling (2006) "Objects First with Java" Pearson.
- ... provided the objects really to behave like the entities in the real world they are supposed to correspond to

## Objects and Maintainability

- if we know the the relevant part of the “real world” (it’s in our “knowledge base”) then objects that correspond to it will be easier to understand
- if the objects correspond to something in the “real world”, then if the real world thing doesn’t change neither should the objects
- if the objects correspond to something in the “real world”, then we should be able to test them the way we would in the real world.

## Finding Good Objects

- Some nouns in the description don't correspond to things we need to build (e.g. "The details of the game (adapted from the Wikipedia article on Kalah) are as follows."
- Some things we need to write code for don't appear in the real world (e.g. pull-down menus).
- Some reasonable-sounding objects appear in designs that are not part of any description of what needs to be built (e.g. HashMap)
- **But**, all advice points to choosing objects that "make sense" with respect to the problem being solved.

## Understanding the problem

- **context schema** "The collection of all of the design agent's beliefs about the context; the design agent's mental picture of the context."
  - e.g. We know that Drafts (also known as Checkers) is played on an 8x8 grid (checkerboard)
- **design schema** "The collection of all of the design agent's beliefs about the diversity of possible design artifacts, including design decisions and candidate solutions; the design agent's mental picture of the design object and its alternatives"
  - e.g. We know that a pull-down menu is a compact way to provide users with choices.

Based on: Paul Ralph (2015) "The Sensemaking-Coevolution-Implementation Theory of software design" Science of Computer Programming. Vol101 `https://doi.org/10.1016/j.scico.2014.11.007`

## Number of Objects and Maintainability

- The closer the classes match concepts in the context schema (and design schema), the better the maintainability
- What does “match” mean? How can we tell that class “properly” matches the concept?
- One aspect to “match” is, the expected number of objects get created
- Example:

  If have a class `Piece` representing the pieces on the board, then expect at most 24 `Piece` objects (12 one colour and 12 a different colour)

## Class Exercise

1. Forms teams of 4-5
2. Create team response to exercise by listing team members
3. Examine the description of the game Noughts and Crosses on Wikipedia (or other descriptions)
4. Identify relevant concepts from the context schema
5. For each concept, provide an appropriate name, a brief description, and the number of objects that should be expected if there were a class for that concept
6. Write your responses in the Lecture 1.2 discussion on Canvas.

## Key Points

- Object-oriented design is creating a program that, when it executes, results in objects sending messages to each other.
- A good object-oriented design has objects that *make sense* with respect to the context schema.
- Classes tell us what objects can be created
- The closer classes match concepts in the context schema, the more maintainable the design
- One aspect of “matches” is, is the number of objects created from the class what we expect from the context schememma